

Cohesion vs Coupling

Cohesion	Coupling
Cohesion represents the relationship within a module – Intra-module.	Coupling represents the relationships between different modules – Inter-module.
Increased cohesion good for software design.	Increased coupling should be avoided during software design.
Cohesion represents the functional strength of modules.	Coupling represents the independence among modules.
Highly cohesive module gives best software.	Loosely coupled modules give the best software.
Six types of Cohesion: • Functional Cohesion • Procedural Cohesion • Temporal Cohesion • Sequential Cohesion • Layer Cohesion • Communication Cohesion	Six types of Coupling: • Common Coupling • Control Coupling • Stamp Coupling • Data Coupling • Content Coupling • External Coupling

Cohesion Types

- **Functional / Logical cohesion:** A subsystem that performs the tasks that are logically related with each other is called logically cohesive.
- **Temporal cohesion:** The subsystem in which the tasks need to be executed in some specific time span is called temporal cohesive.
- **Procedural cohesion:** When processing elements of a subsystem are related with one another and must be executed in some specific order, such subsystems is called Procedural cohesive.
- **Communication cohesion:** when the processing elements of a subsystem share the data then such subsystem is called communication cohesive.
- **Sequential cohesion:** when the output of a particular subsystem is given as input for other subsystem is called Sequential cohesion.

Coupling Types

- **Data coupling:** The data coupling is possible by parameter passing or data interaction.
- **Control coupling:** The modules share related control data in control coupling.
- **Common coupling:** In common coupling common data or global data is shared among the modules.
- **Content coupling:** Content coupling occurs when one module makes use of data or control information maintained in another module.
- **Stamp coupling:** Stamp coupling occurs when modules share a composite data structure and use only parts of it, possibly different parts (e.g., passing a whole record to a function that needs only one field of it).